# Lab 3

## Heterogeneous Cores: Profiling Snapdragon SoC

Qualcomm HDK8850 · Snapdragon 8 Elite Gen 5 (SM8850P) · CPU · GPU · DSP · NPU

## Deadlines and Grading

- **Lab Report (75 pts.):** Due Saturday November 14th, 11:59 PM ET — **in teams of 3**

  There are multiple analysis questions throughout the lab, in Tasks 2 to 12. For all questions, you are expected to flex your analytical reasoning skills and back your ideas with data, where appropriate.

  Turn in your lab report via the Lab 3 Gradescope assignment. The lab report should be properly formatted. We recommend doing the lab report in **LaTeX**, and an example lab report template can be found on Piazza and Canvas.

  - Task 1: Setup (2 pts.)
  - Task 2: The Iron Law Across Three Workloads (5 pts.)
  - Task 3: Measuring the Memory Subsystem (8 pts.)
  - Task 4: Roofline: Place Your Own Workloads (6 pts.)
  - Task 5: Break the Machine on Purpose (4 pts.)
  - Task 6: Explain Every Scaling Curve (6 pts.)
  - Task 7: Your Core Is Not Your Own (5 pts.)
  - Task 8: Two Clusters, One Chip (7 pts.)
  - Task 9: Speaking with no words(6 pts.)
  - Task 10: Make the DSP Earn Its Silicon (5 pts.)
  - Task 11: Move the Data Yourself (6 pts.)
  - Task 12: The Boundary and the Copy (4 pts.)
  - Task 13: Two Knobs, the Compiler, and the Choice (3 pts.)
  - Analysis and Data Presentation Style & Formatting (8 pts.)
- **Lab Code (25 pts.):** Due Saturday November 14th, 11:59 PM ET — **in teams of 3**
  - Task 5: `bp_stress.c` and `rob_stress.c` (4 pts.)
  - Task 10: `student_conv.c` and `student_morph.c` (11 pts.)
  - Task 11: `calc_staged()` (6 pts.)
  - Task 12: `memshare.cpp` (4 pts.)

---

**PSA!**

- **The board gets hot!** In this lab, you will perform measurements on real silicon. A board that has been running a 6-core matmul for 10 minutes does not perform the same as it did when you started. Put idle time between sweeps. If a result surprises you, try remeasuring it with a cold HDK before you write it down.
- **One run is not a measurement.** To account for this variance every result in your report must be a mean over repetitions with a coefficient of variance.
- **Save incrementally.** Sweeps in this lab run for tens of minutes. Write each measurement to a CSV as you get it, not at the end of the loop.

- **Use git.** Similar to previous labs, we recommend version control through a private repository. Your code should NOT be public.
- **Beware of the compiler.** Multiple tasks ask you to purposefully write bad code. If a measurement comes back looking exactly like the baseline, your first hypothesis should be that the compiler removed the thing you wrote from the binary.
- **You are not alone.** Plan ahead to coordinate the board usage and with your partners. Pair programming is encouraged and you should think over all questions together with your team. Knowledge of the entire lab exercise is expected of all students.
- **The profiler.** This is the tool you are going to use throughout the lab, its guide is located at "profiler guide.pdf". It has a lot of helpful information so please use it.

---

**Section I: Overview and Setup**

⇨ Task 1: Setup and the Privilege Trap

**Section II: Single-Core CPU**

⇨ Task 2: The Iron Law Across Three Workloads

⇨ Task 3: Measuring the Memory Subsystem

⇨ Task 4: Roofline: Place Your Own Workloads

⇨ Task 5: Break the Machine on Purpose

**Section III: Multi-Core CPU**

⇨ Task 6: Explain Every Scaling Curve

⇨ Task 7: Your Core Is Not Your Own

⇨ Task 8: Two Clusters, One Chip

**Section IV: The GPU**

⇨ Task 9: Speaking with no words

**Section V: The DSP**

⇨ Task 10: Make the DSP Earn Its Silicon

⇨ Task 11: Move the Data Yourself

**Section VI: The NPU**

⇨ Task 12: The Boundary and the Copy

⇨ Task 13: Two Knobs, the Compiler, and the Choice

**Final Deliverables**

**Appendix**

# Section I: Overview and Setup

The board you will use for this lab is a Qualcomm HDK8850 (Everyone thank Qualcomm for their patronage). It's built around the Snapdragon 8 Elite Gen 5 (SM8850P). It has the following features:

| Engine | Part |
|---|---|
| **CPU** | Oryon-3: **Gold** cores 0–5 up to 3.63 GHz, **Prime** cores 6–7 up to 4.6 GHz |
| **GPU** | Adreno 840: 18 power levels, 160 MHz to 1.2 GHz |
| **DSP** | Hexagon cDSP v81: 8 HVX contexts, 8 MB VTCM |
| **NPU** | Hexagon HTP |

## ⇨ Task 1: Lab Set up

For this task, any of the commands highlighted in blue should be run locally in your computer. Any of the commands highlighted in grey should be run inside your Docker container. Everything highlighted in green is the correct printout you should see.

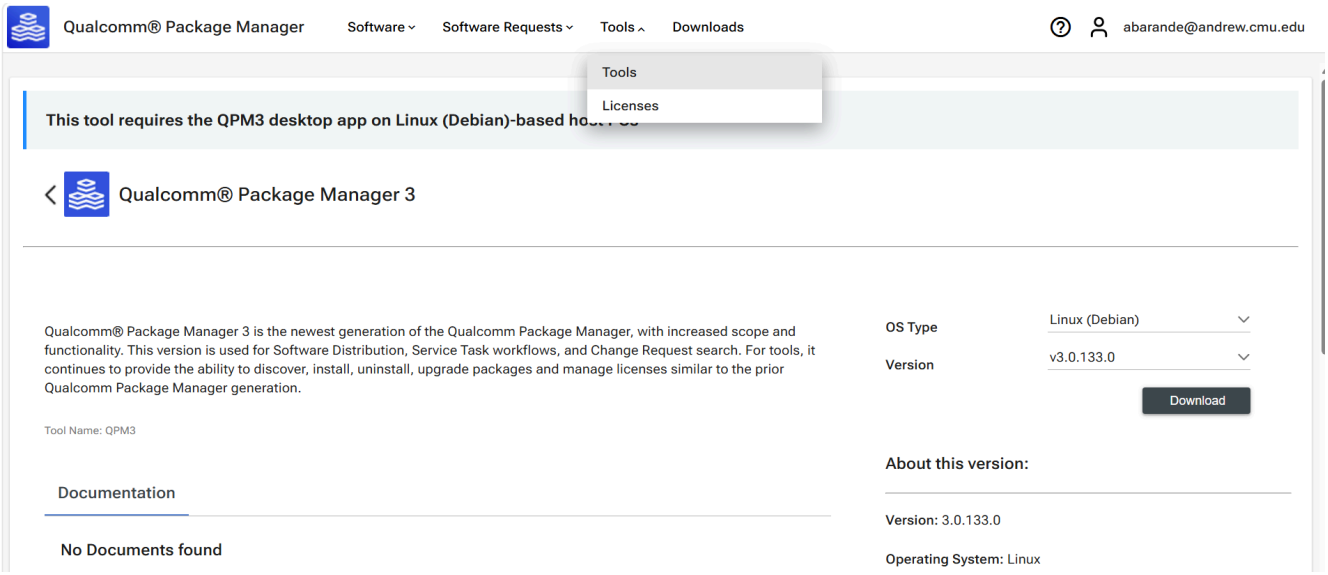
**On your machine**

**Step 1: Install Python and Docker Desktop.**

- **Python 3.10+:** check with `python --version` (`python3` on macOS/Linux). On Windows, tick "Add python.exe to PATH" in the installer. Nothing in this lab is ever installed with pip.
- **Docker Desktop** from `docker.com/products/docker-desktop`. Start it and wait until the engine is running.
  **For Apple Silicon:** Settings → General → tick "Use Rosetta for x86/amd64 emulation".

**Step 2: Get a Qualcomm account and download QPM.**

- Sign up at `myaccount.qualcomm.com`
- At `qpm.qualcomm.com`, open **Tools** in the top menu bar and pick **Tools** from the dropdown. Search "QPM", open Qualcomm Package Manager 3, set OS Type to **Linux (Debian)** and download the .deb (about 260 MB). Download the Linux version no matter what OS your computer/laptop uses.




Screenshot of the qualcomm page where you download the QPM.

### Step 3: Download the source code

Download and extract the bundle from Canvas. On GitHub, create an empty private repository. Don't add a README or any other file. Then inside the extracted folder (the one that contains README.md and profiler.py):

**Your machine**

```
git init -b main
git add -A
git commit -m "Lab 3 starter code"
git remote add origin <your repository URL>
git push -u origin main
```

If git asks for a password, it wants a personal access token (see Step 8). From now on, every local command you run should be run from this folder.

### Step 4: Get adb on your machine.

**Your machine**

```
python3 install.py --fix
. .toolchain/activate.sh #for MacOS/Linux
python install.py --fix
. .toolchain\activate.ps1 #for Windows
```

Re-run the activate line in every new terminal. It puts adb on your PATH.

### Complete the following steps in the Docker container

### Step 5: Create your lab container.

**Your machine**

```
python scripts/lab3.py #python3 on MacOS/Linux
```

*You should see:*

```
created and started: lab3
 ...
 root@a1b2c3:/workspace#
```

*exit* stops the container but doesn't delete it. Run *python scripts/lab3.py* again to get back in

**Never** run *docker system prune*, *docker rm*, or Docker Desktop's "Clean / Purge data".

**Step 6: Install QPM in the container.**

**Your machine**

```
python scripts/lab3.py qpm "~/Downloads/QualcommPackageManager3*.deb" #python3 for
MacOS/linux
```

Change the path if your download is elsewhere. Ignore errors such as *bc: not found* or *Illegal number*. If you reach *Post-Install Complete* then everything is fine.

**Step 7: Install the two Qualcomm SDKs.**

Open a shell in the container with `python scripts/lab3.py`, then run these one line at a time:

**In the container**

```
qpm-cli --login <what you sign in to qpm.qualcomm.com with>
qpm-cli --license-activate qualcomm_ai_runtime_sdk
qpm-cli --extract qualcomm_ai_runtime_sdk -v 2.40.0.251030
qpm-cli --license-activate hexagonsdk6.x
qpm-cli --install hexagonsdk6.x
```

Accept the licenses when asked and put your username.

**Step 8: Clone your code into the container.**

**Your machine**

```
python scripts/lab3.py clone <your repository URL> #python3 for MacOS/Linux
```

The username is your GitHub username and the password is a **personal access token** (github.com/settings/tokens → classic, one scope: `repo`).

More info on access tokens and how to set them up can be found at: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

**Step 9: Install the NDK.**

**In the container**

```
python install.py --role dev --fix.
```

**Step 10: Check your setup.**

**In the container**

```
python scripts/check_setup.py
```

It checks Python, adb, QAIRT, the NDK, the Hexagon SDK and Docker, marking each line [PASS] or [FAIL]. It should end with: SETUP OK.

If something fails with the hexagon-SDK, run *hexagon-doctor*.

**Complete the following steps with the board**

**Step 11: Connect the board.**

Use the USB-C cable provided. First connect the board's power, then connect it to your computer. Check that your machine sees the board, and that the container does too:

**Your machine**

```
adb devices -l
python scripts/lab3.py status
```

### Step 12: Provision the board.

**In the container**

```
python scripts/provision_device.py --qairt-root
/opt/qcom/aistack/qairt/2.40.0.251030 --dsp-skel
kernels/matmul/dsp/prebuilt/libdsp_matmul_skel.so #its one line
```

*The summary should end like this before you can proceed:*

```
  npu_validator     PASS
  npu_fp16_client   PASS (real sustained FP16 graph)
  npu_int8_client   PASS (real sustained INT8 graph)
  dsp_verify        PASS (HVX dispatch, fingerprint verified)
  provenance        PASS
```

If any of those five lines is not a PASS, stop and tell a TA.

### Step 13: Verify the board.

**In the container**

```
python scripts/verify_board.py
```

*You should see:*

```
17 pass, 0 warn, 0 fail, 0 skipped

BOARD OK.
```

This script runs a real dispatch on all four engines, so `BOARD OK` means the lab will work.

### Step 14: Take your first measurement.

**In the container**

```
python profiler.py matmul cpu -p size=512 --core prime
```

*You should see:*

```
matmul.cpu    7160.5 us/iter x10   37.5 GFLOP/s   0.4 GB/s   fp=3.0479e+03
```

Your speed will differ; that is fine. The `fp=` value must match exactly.
Now run it six times using the repeat argument:

**In the container**

```
python profiler.py matmul cpu -p size=512 --core prime --repeat 6
```

*It ends with a summary like:*

```
  n=6  GFLOP/s: mean 36.9  range 33.6-38.6  CoV 5.4%
```

### The development loop for the rest of this lab.

Edit in your own editor, then commit, push, and hand it to the container:

**Your machine**

```
git add -A
git commit -m "conv: tile the inner loop"
```

```
git push
```

**In the container**

```
git pull
bash scripts/build_student.sh conv
python profiler.py convolution dsp
```

Above is an example commit. The development loop is that whenever you write code, you'll have to write it locally, push it to your private repo, then push it to the board. Keep this in mind for every task, and this handout will be explicit where necessary.

**Board rules.**

- Never run `adb root`. (If you did: `adb unroot`.)
- Never flash, factory-reset or adb reboot bootloader the board, and do not delete anything under `/data/local/tmp/profiler-kernels/`. If the board seems broken, tell a TA.

**Task 1.1** Include in your report the last 2 lines of your `verify_board.py` run (the pass count and `BOARD OK`) and the summary line of your `--repeat 6` run (mean, range and CoV).

# Section II: CPU: Single-Core

This section focuses on a single CPU core. Below are the kernels that you will use as benchmarks in this lab:

| Workload | What it does |
|---|---|
| `saxpy` | Computes `y = a·x + y` over a large set of data – 2 loads and a store for every 2 FLOPs |
| `reduction` | Sums the contents of an array and is read-only |
| `convolution` | Performs a direct 2D convolution: you can control its arithmetic intensity through the channel count |
| `matmul` | Performs matrix multiplication – dens $O(N^3)$ arithmetic on $O(N^2)$ data |

```
python profiler.py --catalogue          # every workload, variant and parameter
python profiler.py --list-counters      # every hardware counter, by name
```

## ➪ Task 2: The Iron Law Across Workloads

Recall from lecture, that the Iron Law frames execution time as the product of **instruction count**, **CPI**, and **clock cycle duration**. In this task, you will explore how each of these terms affects performance in real hardware.

Run `convolution`, `matmul` and `saxpy` on one Prime core at six clock frequencies spread across the cluster's frequency range. **Report in your lab report the frequency ranges you chose.** At each frequency, record wall time, IPC, `effective_ghz` and the backend memory-stall counter. The stalls counter preset reports all of these metrics/counters:

```
python profiler.py matmul cpu -p size=1024 --core prime --counters stalls --repeat 3
```

Set the clock with `scripts/power_pin.py`, using only frequencies from the board's own list:

```
adb shell cat /sys/devices/system/cpu/cpufreq/policy6/scaling_available_frequencies
python scripts/power_pin.py prime 1382400   # kHz; must appear in the published list
python scripts/power_pin.py show            # confirm that the pin took effect
python scripts/power_pin.py restore         # restore the board when finished
```

If you use a value not in the frequency range, the write will succeed but the clock will not change.

**Task 2.1** Run the following two commands:

```
python profiler.py matmul cpu -p size=1024 --core prime --repeat 10
python profiler.py matmul cpu -p size=1024 --core prime --repeat 10 --variant eigen
```

`--repeat` runs the workload as separate processes and prints the mean, range, and CoV. Repeat for `size=512`.

Compare the means and the spreads. Explain what they tell you about where the variance comes from? Compare the CoVs at 512, look at how long the runs take. Which variant is noisier, and why?

Based on these results, pick the shortest run length you'd trust a performance number from, keep this cutoff for the rest of your lab. Write in your lab report for this task the length you picked and justify based on your results.

**Task 2.2:** With counters on, the profiler reports two clock numbers: `scaling_cur_freq_ghz`, the highest frequency the governor (the part of the OS that sets the clock speed) asked for while the workload ran, and `effective_ghz`, derived from the cycles the core actually counted. Explain what `effective_ghz` shows that is not apparent with `scaling_cur_freq_ghz`.

Run:

```
python profiler.py matmul cpu -p size=512  --core prime --variant eigen --counters basic
python profiler.py matmul cpu -p size=2048 --core prime --variant eigen --counters basic
```

Report both values at each size, explain why the two sizes behave differently. Between `effective_ghz` and `scaling_cur_freq_ghz` which would you put in a performance report and why.

**Task 2.3:** Run these three at each of your six frequency setpoints. Keep the settings exactly as written at every clock, so the clock is the only thing that changes:

```
python profiler.py matmul cpu -p size=1024 --core prime --counters stalls --repeat 3
python profiler.py convolution cpu --core prime --counters stalls --repeat 3
python profiler.py saxpy cpu -p size=16777216 -p iterations=100 --core prime --counters
stalls --repeat 3
```

Plot normalized runtime (`t` / `t` at `f_max`) against normalized inverse frequency (`f_max` / `f`) for all three, on one set of axes, with `f_max` as your highest **pinned** setpoint. Say which is limited only by the clock. *Hint: A workload whose runtime is purely clock-limited lands on the diagonal.*

**Task 2.4:** For the outlier, IPC rises as the clock falls. Which term absorbs part of the frequency change, and what physical quantity stays the same while the clock changes? You need to use the backend stall counters (`backend_stall_pct`, `slot_backend_pct`) as support for your justification.

**Task 2.5:** A teammate wants to run the outlier at half the maximum clock and says it will barely slow down. Test the claim by running the outlier at half the maximum clock and reporting what happens. Change its size so it fits in the cache, at both clocks, measure it and report your results. Explain if the claim is valid.

## ⇨ Task 3: Measuring the Memory Subsystem

Sometimes life will not give you the datasheet or the cache sizes you need. So you will now measure them yourself! We made a tool for you called cachelat:

```
python profiler.py cachelat cpu -p kb=64 --core prime --counters
raw-l1d-cache,raw-l1d-cache-refill,raw-l1d-cache-hwprf
```

It reports `ns_per_access`, the load-to-use latency.

Some useful parameters:

`-p pattern=random` visits the lines in a shuffled order meant to confuse the prefetcher.
`-p pattern=sequential` visits the same lines the same number of times, in address order.
`-p smallpages=true` forces 4 KB pages. (The board uses 2 MB pages by default)

More information on useful parameters or counters can be found on the profiler handout.
**Pin the clock before you sweep anything.**

**Task 3.1:** Report the L1D$ and L2$ sizes for the Gold and the Prime cluster, use evidence to back your answer. *Hint: Make sure the prefetcher is not hiding a boundary from you*.

**Task 3.2:** Sweep the working set again using `-p pattern=sequential`. What does this latency tell you about whether the data still fits in the L1$? Which counter shows the L1$ boundary more clearly? How does its change compare with the change in latency?

**Task 3.3:** Run `pattern=sequential` and `pattern=random` on the same buffer, at a working set much larger than L2$. Report the per-access latency of each, and what share of the L1D$ accesses came from the prefetcher. Explain the gap. Can the core overlap loads in either pattern?

**Task 3.4:** Not every knee in a latency curve is a cache. Repeat your random sweep with `-p smallpages=true` and compare it with your default runs. Where does page size make the biggest difference? For each place it matters say if it's the TLB or the cache, back your answer with counters.

**Task 3.5:** Even before you time anything, which counter predicts how much 2 MB pages will help? Why?

## ⇨ Task 4: Roofline: Place Your Own Workloads

In this Task you'll calculate the roofline of this board and place the kernels we gave you onto it.

Measure the roofs on one Prime core. Do not use datasheet numbers.

`python profiler.py --catalogue` gives you every workload with its variants and parameters.

**Task 4.1:** Derive the arithmetic intensity of all 4 kernels. State your assumptions (a cold cache, whether stores count, and whether a value read twice counts twice , etc)

**Task 4.2:** Measure the compute roof and the memory roof yourself. Use whichever kernels and settings give you the highest FLOP rate and the highest bandwidth. State which you used for each roof. Then place each kernel on your roofline at its arithmetic intensity and include the plot.

**Task 4.3:** For each kernel, would wider vector units or more memory bandwidth help more? Use your roofline to explain why.

**Task 4.4:** Your roofline model accounts for 2 distinct upper bounds. Name one effect measured in Task 3 that this model doesn't consider. Estimate how this effect could be included in your model.

**Task 4.5:** Explain why we can not use the datasheet numbers effectively here.

## ⇨ Task 5: Break the Machine on Purpose

Angry at this lab and want to smash the board? Too bad. Try to make it break with your *Programming Skills*.

You'll write 2 small programs, each trying to make one metric as bad as you can.

Your code is in kernels/lab3/bp_stress/bp_stress.c and kernels/lab3/rob_stress/rob_stress.c. After every edit, push it to your repo, pull it inside the docker, then *`python scripts/new_kernel.py <name> --push`* inside the container.

Read the PSA about the compiler again before you start. Follow the instructions in the set up to get your code into the container and upload it into the board.

**Task 5.1 (bp_stress.c):** Write a program with the highest branch misprediction rate you can get. Measure it as *`raw-br-mis-pred / raw-br-pred`*:

```
python scripts/new_kernel.py bp_stress --push    # after every edit
python profiler.py custom cpu --binary bp_stress --core prime --counters raw-br-pred,raw-br-mis-pred
```

12% mispredicts get full credit and 20% is the stretch goal. Report your method and your rate. Use *`–counters branch`* to explain why your method was right.

**Task 5.2:** On your worst case, collect 4 counters: raw-br-mis-pred and raw-br-mis-pred-retired (mispredicts, counted speculatively and at retirement), and raw-br-pred and raw-br-retired (all branches, counted the same way). What do you observe? Explain.

**Task 5.3 (rob_stress.c):** Write a program that maximizes backend stalls.

```
python scripts/new_kernel.py rob_stress --push
python profiler.py custom cpu --binary rob_stress --core prime --counters stalls
```

75% *`backend_stall_pct`* (stalled cycles) gets full credit, and 95% is the stretch goal. Report your method and your results.

**Task 5.4:** Compare your cycle-based *(`backend_stall_pct`)* and slot-based *(`slot_backend_pct`)* stall numbers. How do they count stalls differently? Which counter better describes lost throughput?

Use *`raw-stall-slot`* and *`cpu-cycles`* (both in *`--counters stalls`*) to work out the core's issue width. Use *`mem_stall_pct`* to show one configuration of your program whose stalls are mostly waiting on memory and one whose stalls are mostly waiting on dependencies.

# Section III: CPU: Multi-Core

As mentioned before, your CPU has 8 cores, not just one. Now you will get to use all of them!

## ⇨ Task 6: Explain Every Scaling Curve

Run *saxpy, reduction, convolution and matmul* with 1 to 6 threads on the Gold cluster:

```
python profiler.py saxpy cpu -p size=16777216 -p iterations=100 --core gold --threads 1
--repeat 3
python profiler.py reduction cpu -p size=16777216 -p iterations=120 --core gold --threads 1
--repeat 3
python profiler.py convolution cpu -p iterations=150 --core gold --threads 1 --repeat 3
python profiler.py matmul cpu -p size=1024 -p iterations=10 --core gold --threads 1
--repeat 3
```

**Task 6.1:** Some workloads do not have a noticeable speed up. For each of those, find the cause: change one variable (data size, clock, etc.) while keeping everything else the same, and rerun. Report what you changed, what you expected, and what happened.

**Task 6.2:** Measure conv's IPC as you add threads, from 1 to 6:

```
python profiler.py convolution cpu -p iterations=150 --core gold --threads 1 --counters
stalls
```

What does flat IPC tell you? Think about what it means for shared resources. Look at the 5- and 6-threads run, what happens to the time per iteration, and what happens to IPC? Explain using other counters as support. *Hint: remember that power_pin.py can hold the clock fixed.*

**Task 6.3:** Explain matmul's behaviour. Why is its speedup on 6 threads so far below 6×? Run it at *-p size=512* as well as 1024:

```
python profiler.py matmul cpu -p size=512 -p iterations=60 --core gold --threads 1
--counters stalls
```

Use the Iron Law to show how much of the gap each term explains. *Hint: look at effective_ghz and IPC.*

**Task 6.4:** Fit Amdahl's Law to *conv* and to *matmul* using only your 1–4 thread results. What serial fraction does each fit give, and what does it predict at 6 threads and how does it compare to your measurements? Plot parallel efficiency against threads for both. How does Amdahl predict efficiency should change as you add threads, and what does your data do instead?

## ⇨ Task 7: Your Core Is Not Your Own

You have one Prime core to yourself. A colleague claims *nothing running elsewhere on the chip can affect your measurements.* Test this claim.

Run your victim on core 7. Prime and gold clusters each have their own L2. It is up to you to decide which workload to be your aggressor, how many and in which cores.

Use *run_parallel.* An example on how to use it:

```
from profiler import Profiler, Backend, reduction, saxpy
p = Profiler()
victim = reduction(Backend.CPU, size=16777216, iterations=300, core=7, counters="stalls")
agg1   = saxpy(Backend.CPU, size=16777216, iterations=150, core=6)
alone = p.run(victim)
both  = p.run_parallel([victim, agg1])[0]      # victim's result comes first
print(alone.gbps, both.gbps, alone.gbps / both.gbps)
print(both.derived["ipc"], both.derived["effective_ghz"]) # printing the victim counters
```

You can add more aggressors like this: p.run_parallel([victim, agg1, agg2…..]). The aggressor has to have a longer run than the victim. So if its `wall_us` is not larger, raise its iterations.

**Task 7.1:** Report the worst slowdown you caused. Explain your set up (victim and aggressor workloads, which cores, etc.).

**Task 7.2:** Which hurts the victim more, one aggressor on core 6, or six aggressors on cores 0–5? Measure both and explain this result. Based on this, which shared resource is the most contended?

**Task 7.3:** Find and report a configuration with six busy aggressor cores that barely slows the victim down. Explain why the slowdown is small. Use counters to explain where the slowdown comes from.

**Task 7.4:** Now make the victim `matmul` at size 512 on core 7 with a memory-hungry aggressor on core 6. Report the victim's time, IPC and cache miss rates (use `counters="cache"`) with and without the aggressor. Then design and run an experiment that proves which shared resource is slowing the victim down. *Hint: look at the behaviour of the miss rates*

## ⇨ Task 8: Two Clusters, One Chip

Marketing says this SoC has eight Oryon cores in two clusters. A colleague claims the clusters use identical cores and differ only in maximum clock. You will have to prove or disprove their claim.

### Part A — Are they the same core?

To compare the two clusters fairly, run them at the same clock. Check which setpoints match in their respective frequency ladders found in `/sys/devices/system/cpu/cpufreq/`. Policy 0 for core 0 and so on. You will need to run them at that frequency.

**Task 8.1:** One at a time, run each of the four workloads once on core 0 and once on core 7. Compare their IPC and runtime. Is your colleague right? Is the gap the same for every workload? Explain.

**Task 8.2:** Find a workload that shows Prime having higher IPC and backend-stall percentage than Gold. Explain how both can be simultaneously true of the same core.

### Part B — Where should the work go?

**Task 8.3:** For each of the workloads, find the core mask that gives the shortest run time (set *threads* to the number of cores in the mask). Report the best mask, and the ratio between it and the worst one. If a workload's best mask isn't all eight cores, find which core is best left idle. What do you gain by leaving it idle? Explain using Task 4's rooflines. *Let the board cool between runs*

**Task 8.4:** The kernels split work into equal bands, one per thread. Since not all cores are the same, is that still the best solution? Why or why not? Use your 8.1 IPC numbers and the clock ratio to predict a better Prime-to-Gold work ratio.

**Task 8.5:** Matmul lets you give Prime threads a bigger share of the work using `-p prime_weight=W`. Sweep W and find the best result:

```
python profiler.py matmul cpu -p size=1024 -p iterations=30 --core all --threads 8 -p pin=true -p prime_weight=1.8 -v
```

How much faster is it than the even split? Compare it to your prediction, if it missed, explain why.

## Part C — What does the fastest configuration cost?

`p.power()` estimates whole-board power for a task (see the profiler guide).

**Task 8.6:** Run `matmul` on one Prime core at its top clock, 4.3968 GHz. Write down its throughput. Then find the lowest Gold clock at which six Gold threads match or beat it. Report them.

**Task 8.7:** For both configurations you just used, measure `p.power()`. Report energy per unit of work (*power_w* divided by throughput, J/GFLOP). Which setup uses less energy? Compare that with the total core·GHz each one spends, and explain the result.

**Task 8.8:** Run both configurations on `SAXPY`. Compare them by energy per unit of work (*power_w* divided by throughput in GB/s, from *r.gbps*). Which set up uses less energy? Think about your `SAXPY` results in part B and explain your answer. Why is it different from your `matmul` results?

# Section IV: The GPU

This GPU sadly does not give you any hardware counters. Thankfully, you are a super smart 740 student! When you can't read a number directly, you sweep a parameter it depends on and read the hardware off the shape of the curve.

Everything in this section uses one binary, *`gpu_memlab`*:

```
python profiler.py gpu_memlab gpu -p stride=1 -p kb=65536 -p reps=200
```

Read the profiler handout for more information on it.

## ⇨ Task 9: Speaking with no words

### Part A — Measuring the cache line

**Task 9.1:** Sweep the stride between neighboring work-items (*`-p stride=`*, counted in 4-byte floats) starting from 1 and doubling until 64. Plot achieved bandwidth against the number of bytes between them. Report your cache line size and the feature of the curve you read it from. Explain why that feature marks the line size.

**Task 9.2:** At larger strides the curve falls again. Why do you think this happens? Design and run an experiment to test it, one whose result would come out differently if your cause were wrong. *Hint: You can vary kb (power of two, in KiB), global (the number of work-items) and the stride.*

**Task 9.3:** You just saw that wasting most of each cache line costs time. Does it also cost energy? Measure stride 1 and 16 with *`p.power()`*, report the energy per useful byte(power_w/r.gbps). Is the difference of power bigger than that of bandwidth? Is wasted bandwidth a speed problem, an energy problem, or both?

### Part B — The cost of a useless lane

**Task 9.4:** Run *`-p mode=divergence`* and sweep *`-p active=`* from 64 down to 1, halving each time. Record the execution time and the throughput at each point. Explain what you see.

**Task 9.5:** Now sweep the workgroup size with *`-p mode=divergence -p active=64 -p local=N`*, with N from 8 up to 256, doubling. Report where the growth stops. How many lanes does the GPU run together? Does this agree with 9.4?

**Task 9.6:** Give a realistic workload where lanes that run together would often take different paths, and describe one algorithmic change that reduces the problem without changing what the workload computes.

## Part C — When is sparse actually faster?

A colleague argues that a 4000 × 4000 matrix with 40% zeros (density 0.6) should run about 1.7× faster in CSR sparse format `(spmv)` than dense `(densemv)`, because that's how much arithmetic it skips.

**Task 9.7:** Test the claim. Find the density at which sparse stops beating dense, and explain the gap using what you learned about cache lines in Part A.

**Task 9.8:** Another colleague repeats your experiment at 4096 × 4096 and gets a very different result. Verify it yourself. Explain the changes. Which matrix size gave you an answer you'd trust? Find a way to lay out a 4096 × 4096 matrix in memory so `densemv` runs at most at 1.5X the time it had at 4000 × 4000. Show proof.

# Section V: The DSP

In this section you will be comparing one Hexagon core using all its HVX contexts against one Prime CPU core with `--threads 1`.

## ➪ Task 10: Make the DSP Earn Its Silicon

### Part A — Make the DSP use its hardware

First you will greet the DSP. `dsp_hello_test` pings it and prints how many HVX contexts it has and how much VTCM. Record both:

```
adb shell "cd /data/local/tmp/profiler-kernels/dsp && LD_LIBRARY_PATH=.
ADSP_LIBRARY_PATH='.;\$PWD' ./dsp_hello_test"
```

Now take both baselines on the same convolution shape:

```
# CPU baseline, one Prime core
python profiler.py convolution cpu -p H=256 -p W=256 -p cin=64 -p cout=64 -p k=3 -p
iterations=5 --core 7 --threads 1 --repeat 3

# DSP baseline: build the unmodified kernel, push it, measure
bash scripts/build_student.sh conv
python profiler.py convolution dsp -p H=256 -p W=256 -p cin=64 -p cout=64 -p k=3 -p
iterations=5 --repeat 3
```

Note each run's GFLOP/s and its fingerprint (`fp=`). Your modified kernel must give the same fingerprint.

You will use `kernels/lab3/conv/student_conv.c`. **Change MY_THREADS and my_range()** so the rows are split across all HVX contexts, then rebuild, push and measure again.

**Task 10.1:** Report your speedup over the unmodified kernel. Why isn't it equal to the number of HVX contexts? What resources do context share that CPU cores do not? Report your parallel efficiency at a small context count vs at the max, explain the changes.

### Part B — Find where the DSP wins

With your optimized kernel, compare one Hexagon core against one Prime core (--core 7 --threads 1) on these shapes, all with k=3:

- FP32: 256×256 C=32, 256×256 C=64, 256×256 C=128, 128×128 C=128, 128×128 C=256
- INT8: 128×128 C=128, 128×128 C=256

For example, for 128x128 C=256:

```
# FP32
python profiler.py convolution cpu -p H=128 -p W=128 -p cin=256 -p cout=256 -p k=3 --core 7
--threads 1
python profiler.py convolution dsp -p H=128 -p W=128 -p cin=256 -p cout=256 -p k=3

# INT8 (prints GFLOP/s, but these are integer operations: read it as GOP/s)
python profiler.py convolution cpu --variant int8 -p H=128 -p W=128 -p cin=256 -p cout=256
-p k=3 --core 7 --threads 1
python profiler.py convolution dsp --variant int8 -p H=128 -p W=128 -p cin=256 -p cout=256
-p k=3
```

**Report the DSP speed, the CPU speed and the ratio for every shape.**

**Task 10.2:** Which shape gives the DSP its biggest advantage, and which does it still lose? Give both ratios.

**Task 10.3:** As the problem grows, one processor gets faster and the other gets slower. Which is which, and why?

**Task 10.4:** Explain your best DSP result using the Iron law. The clock is given to you: the Hexagon runs near 1 GHz and the Prime core at 4.4 GHz.

An HVX register is 1024 bits wide and a NEON register (CPU) is 128 bits. How many more operations does one HVX instruction do than one NEON instruction?

Using that and the clock difference, how much faster should the DSP be? Compare that with your measured ratio and explain where the extra speed comes from.

**Task 10.5:** At 128×128 with `-p cout=128`, sweep `-p cin=` through every value from 120 to 136. What do you find, and how big is the jump? Explain it with the HVX register width and the data type. What does this mean for a real model whose channel counts someone else picked?

## Part C — The whole chip

Run all eight CPU cores and all DSP contexts simultaneously:

```
bash scripts/run_concurrent.sh
```

Run it three times and report every run, plus the mean and range of each slowdown.

**Task 10.6:** Report the throughput of the CPU by itself, the DSP by itself and both combined. How does each engine change when running together? How much does adding the DSP cost the CPU?

**Task 10.7**: Should a design review compare one DSP against one CPU core, or against all eight? Explain what is useful about each comparison. What other measurement would help you settle this decision?

**Task 10.8:** If the CPU is completely busy, what is the DSP worth? Explain using your results from 10.6. Based on what you have seen so far, what kind of workload would you not put on the DSP? Why?

## Part D — Repairing an image

We are now going to see what the dsp is actually useful for. You are going to play with two simple image operations: **Erosion** replaces each pixel with the smallest value in its neighborhood, and **dilation** with the largest.

We are giving you a damaged picture in: `kernels/convolution/dsp/data/dog_corrupted.png`

**Task 10.9:** Look at dog_corrupted.png and say whether Erosion or Dilation would improve its quality. Explain why and what would the other one do. Use the following to test:

```
python profiler.py morph dsp -p impl=qhl -p op=erode -p write_pgm=true
python profiler.py morph dsp -p impl=qhl -p op=dilate -p write_pgm=true
#run this next part in your laptop, not in the container
docker cp lab3:/workspace/morph_images .  #to copy the images to your laptop
```

Please do not forget to run the last command so you can see your results! I bet the pictures made you smile. Report both residuals and include both _compare.png images.

**Task 10.10** Implement both filters in *kernels/lab3/morph/student_morph.c*. Build it and test it:

```
bash scripts/build_student.sh morph
python profiler.py morph dsp -p op=erode -p write_pgm=true
```

Your residual must match the QHL residual you got in 10.9. Don't forget to copy your images to your laptop again so you can admire them!

**Task 10.11:** Time your version and Qualcomm's library (QHL) at window **sizes 3, 5 and 7:**

```
python profiler.py morph dsp -p k=3
python profiler.py morph dsp -p k=3 -p impl=qhl
```

Report all six times and the QHL speedup at each size.

**Task 10.12:** You can see that your version's time and QHLs act differently. How and why? Using this, explain what the QHL is limited by. *Hint: The image is 1024×768 pixels at one byte each. Divide the bytes QHL must move (read the image once, write it once) by its time*

## ⇨ Task 11: Move the Data Yourself

The DSP has a small, very fast scratchpad called VTCM. Unlike a cache, there won't be someone to manage this memory for you. You will have to do it yourself!

### Part A — Characterize the scratchpad

We are giving you a new tool called *conv_vtcm_probe*. It will be useful to find out how much VTCM there is and how fast it moves:

```
adb shell "cd /data/local/tmp/profiler-kernels/dsp && LD_LIBRARY_PATH=.
ADSP_LIBRARY_PATH='.;\$PWD' ./conv_vtcm_probe"
```

**Task 11.1:** The probe prints how much VTCM the device claims, then tries real allocations. Does the claim hold? What's the largest single allocation that works, and can two allocations together reach the same total?

**Task 11.2:** Report the VTCM bandwidth from the section 4's table of the probe output. In *kernels/convolution/dsp/src/dsp_conv_vtcm_probe.c*, find what is timed and how the read loop stops the compiler from deleting it.

**Task 11.3:** The last two columns of section 4's table measure an ordinary (malloc) buffer. Plot them next to the VTCM columns. At what size does the ordinary-memory read curve drop, and what does that tell you about the DSP's cache?

**Task 11.4:** The Task 10 convolution always copies its weights into VTCM first. Compute Weight size for C = 64, 128 and 256 (Cin = Cout = C). Using the cache size from 11.3, when would staging the weights actually help? Why? *Hint: Weight size = Cout × K × K × Cin × 4 bytes, with K = 3.*

## Part B — Beat the DDR path

This workload processes 32 MB of data, but VTCM holds only 8 MB, so the data has to go in blocks. `p` sets how many passes the computation makes over each block. There are three versions of it: `ddr` computes straight from ordinary memory, `naive` copies each block into VTCM with memcpy first, and `staged` is what you are going to work on now. It lives in *kernels/lab3/vtcm/student_vtcm.c*.

```
python profiler.py stage dsp -p impl=ddr -p p=1      # or naive, or staged
bash scripts/build_student.sh vtcm                   # after you edit your file
```

Each run reports total time (copy and compute) and a checksum. For a given p, the three versions' checksums must match.

**Task 11.5:** Run `ddr` and `naive`. Report both times and their copy/compute split. Compared with your 11.3 numbers, what does naive's bandwidth tell you about `memcpy`?

**Task 11.6:** Replace `memcpy` in `my_copy()` with an HVX vector copy. Report the new copy bandwidth and total time. Does staged beat `ddr`? Why or why not?

**Task 11.7:** Now copy with the DMA engine instead. `t5_copy()` in `dsp_conv_vtcm_probe.c` shows how. Report the bandwidth and total time. Before each DMA, that code cleans the source out of the cache.Remove the clean and run with `--repeat 3`. What changed and why?

**Task 11.8:** Split VTCM into two buffers: while you compute on one, DMA the next block into the other. Run `-p` p=1, 2, 4, 8 and 16 for both `staged` and `ddr`. Report total time and copy_us. Compare your results with the 11.7 results. How much of the copy did the overlap hide, and what part of it can't be hidden?

**Task 11.9:** At p=1 there is no data reuse, yet staging still wins. Using your numbers, explain why.

**Task 11.10:** VTCM and DMA do by hand what two pieces of CPU hardware do automatically. Who are they? What are the advantages and disadvantages of doing this manually?

# Section VI: The NPU

To program the NPU you use computation graphs. QNN finalizes it for the NPU HW and then you run it.

## ⇨ Task 12: The Boundary and the Copy

### Part A — What the boundary costs

Run `matmul` on the NPU at three sizes:

```
python profiler.py matmul npu -p size=256  --repeat 3
python profiler.py matmul npu -p size=512  --repeat 3
python profiler.py matmul npu -p size=1024 --repeat 3
```

Each line reports execution time, GFLOP/s, finalize_us (how long compiling took) and execute_us_cov_pct (the spread across the 16 runs inside one process). Record your outputs.

**Task 12.1:** Matrix multiply does $N^3$ arithmetic on $N^2$ data. How much does the time grow from 256 to 512, and from 512 to 1024? How does the NPU spend its time at these sizes? *Hint: Compare with what $N^3$ and $N^2$ scaling would predict*

**Task 12.2:** Repeat the sweep with `-p perf_mode=default`. How do the times, growth factors and spread across runs (CoV) change compared with 12.1? What does that do to your 12.1 conclusion?

**Task 12.3:** At N=512, how does `finalize_us` compare with the execution time? In deciding between using the CPU or the NPU, when should finalize count, and when can you ignore it?

### Part B — Making the copy disappear

Your file to work is `kernels/lab3/memshare/memshare.cpp`. Currently, it hands out ordinary memory, and the harness refuses to use it. Your job is to implement the following functions:0:

```
void* AllocateSharedBuffer(size_t size);   /* NULL on failure */
void  FreeSharedBuffer(void* ptr);
int   GetSharedBufferFd(void* ptr);        /* -1 if not shareable */
```

Set `MEMSHARE_STUB_ONLY` to 0 before running anything else.
Build, then compare a baseline run against a shared one:

```
bash scripts/build_student.sh memshare
python profiler.py matmul npu --variant spike -p size=1024
python profiler.py matmul npu --variant spike -p size=1024 -p memshare=./libmemshare.so
```

Your shared run must show `shared_impl=memshare`, and its fingerprint must match the baseline's.

**Task 12.4**: Run both at sizes 256, 512 and 1024 and tabulate the speedup. Explain its growth. And why is there almost none at 256?

**Task 12.5:** Why can the NPU use `rpcmem` memory but not ordinary malloc memory? Think about the difference between `rpcmem` pages and malloc pages.

**Task 12.6:** The harness puts all three tensors in one shared allocation instead of three separate ones. Why? When would three separate allocations be the better choice?

**Task 12.7:** In 12.1 the NPU's time was mostly fixed cost and moving data. Shared memory removed a copy, yet the speedup is only 1.5×. Where is there still moving data, even with shared memory?

## ⇨ Task 13: Two Knobs, the Compiler, and the Choice

You've seen two ways to speed up our `matmul` without changing the algorithm: the power mode and a narrower datatype. To start, predict which one will help more, and how would N affect the difference.

Run the `int8` version in the same three sizes you did `fp16` previously.

```
python profiler.py matmul npu -p size=512 --variant int8 --repeat 3
```

**Task 13.1:** Using your 12.2 results and these, explain which advantage shrinks with N and which grows? Int8 halves the bytes in every tensor, so why is its speedup nowhere near 2×?

**Task 13.2:** If you could ship only one of these two optimizations in a product, which would you choose, and what would you be giving up?

**Task 13.3:** Run N=512 with `--repeat 5`. Is `finalize_us` noisy or stable? How does it change with N? Is *finalizing* random system noise or real work? *Hint: you should also use your 12.1 values to support your answer.*

**Task 13.4:** How many NPU runs (K) does it take to repay `finalize`, compared with doing the same N=512 `matmul` on the CPU? Make an equation comparing total CPU time with total NPU time (finalize included), then compute K against 1 CPU core and against all 8:

```
python profiler.py matmul cpu -p size=512 -p iterations=40 --core 7 --threads 1 --repeat 3
python profiler.py matmul cpu -p size=512 -p iterations=150 --core all --threads 8 --repeat
3
```

**Task 13.5:** Perform the same calculations for `SAXPY` using 65,536 elements, against one CPU core:

```
python profiler.py saxpy cpu -p size=65536 -p iterations=50000 --core 7 --repeat 3
python profiler.py saxpy npu -p size=65536
```

What K do you get? Is there a break-even point? Why or why not?

**Task 13.6:** Check `SAXPY` finalize at 32,768 and at 65,536 elements, write down how long it took on each. Try different values between them to find the exact element count where the jump occurs. List every size you tried, in order, with its `finalize_us`.

**Task 13.7:** Across the jump you found, compare `finalize_us` and the execution time. How much does each change and what does that tell you about where the cliff comes from?

**Task 13.8:** There's a deployed model accepts dynamically changing input shapes. Given what you measured, what problems could this cause? Propose two ways to deal with it, and what each one costs.

**Task 13.9:** Run the N=512 `matmul` on the GPU and compare with your NPU result from 12.1:

```
python profiler.py matmul gpu --variant best -p size=512 --repeat 3
```

Which is faster per run? Assuming `finalize_us` is the NPUs only setup cost, how many runs (K) until it comes out ahead?

**Task 13.10:** You just assumed that `finalize_us` is the NPUs only setup cost, but is this truly the case? You also noticed that the GPU reports no setup at all (no finalize). You want to check if this is true. Design an experiment that measures each engine's full one-time cost using only the profiler, then run it and recompute K.

Explain your method, show the measurements, and say how much your new K differs from 13.9's and why. Finally it's time to make a choice, which engine would you deploy for this `matmul`, and why?

*Tools you can use*: `-v` prints `host_wall_us` (only for single runs), the time the whole process took. `-p reps=N` (NPU) and `-p inner_reps=N` (GPU) set how many runs happen inside one process.

# Final Deliverables

**Report.** One PDF per team, submitted to the Lab 3 Gradescope assignment.

**Code.** Inside the container, run:

```
python make_final
```

Then from your laptop, where you first cloned the folder, run:

python scripts/lab3.py save

This will give you the following final_lab3.zip:

| File | Task |
|---|---|
| kernels/lab3/bp_stress/bp_stress.c, kernels/lab3/rob_stress/rob_stress.c | 5 |
| kernels/lab3/conv/student_conv.c | 10 |
| kernels/lab3/morph/student_morph.c | 10 |
| kernels/lab3/vtcm/student_vtcm.c | 11 |
| kernels/lab3/memshare/memshare.cpp | 12 |

Please submit it to gradescope with your report.

# Appendix

**The profiler guide.** This should be your guiding light, it's located in the */docs* folder. It contains all information about the profiler, including flags, parameters, variants, the Python API and power estimates.

Two commands worth getting familiar with:

```
python profiler.py --catalogue       # every workload, variant and parameter
python profiler.py --list-counters   # every counter, with what it measures
```

**When something breaks**

This script will help you check the connection, the installed kernels, clocks and temperature, and does one run on every engine:

```
python scripts/verify_board.py
```

Some symptoms:

| Symptom | Likely cause and fix |
|---|---|
| no devices or device not in “device” state | Charge-only cable, or the board is off |
| GPU runs fail after adb root | GPU kernels must run as the normal shell user. Run adb unroot |
| DSP runs hang or print nothing | The DSP connection is stuck. Rerun verify_board.py, then reboot the board |
| Numbers drift downward over a long session | The board is hot. Let it idle a few minutes and measure again |
| A change to your DSP or NPU code seems to do nothing | The board is still running your old build. Rerun build_student.sh, and check it says “Pushed to the board” |

If the fixes don't work don't hesitate to reach out to a TA (during business hours)!